



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES





Exp :1 Write the Python Program to solve 8-Puzzle Problem.

Aim:

To write a python program to solve the 8-puzzle problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.

Step2: If the dequeued state is the goal state, return the solution path.

Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its neighboring tiles.

Step5: For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before.

Program:

```
from queue import PriorityQueue
goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)
def heuristic(state):
    in range(3):
        for j in range(3):
            tile = state[i * 3 + j]
            if tile != 0:
                x, y = (tile - 1) // 3, (tile - 1) % 3
                distance += abs(x - i) + abs(y - j)
    return distance
def solve(initial_state):
    frontier = PriorityQueue()
    frontier.put((heuristic(initial_state), initial_state))
    explored = set()
    while not frontier.empty():
        _, state = frontier.get()
        if state == goal_state:
            return True
        for successor
```



```
in successors(state):            if
    successor not in explored:
        priority = heuristic(successor) + len(explored)
    frontier.put((priority, successor))    return False
def
successors(state):    successors = []    i = state.index(0)
    if i % 3 != 0:        # Slide tile to the left        new_state
        = list(state)
        new_state[i], new_state[i - 1] = new_state[i - 1], new_state[i]
    successors.append(tuple(new_state))    if i % 3 != 2:
        # Slide tile to the right
    new_state = list(state)
        new_state[i], new_state[i + 1] = new_state[i + 1], new_state[i]
    successors.append(tuple(new_state))    if i // 3 != 0:        # Slide
    tile up        new_state = list(state)
        new_state[i], new_state[i - 3] = new_state[i - 3], new_state[i]
    successors.append(tuple(new_state))    if i // 3 != 2:
        # Slide tile down
        new_state = list(state)
        new_state[i], new_state[i + 3] = new_state[i + 3], new_state[i]
    successors.append(tuple(new_state))    return successors
initial_state = (2, 8, 3, 1, 6, 4, 7, 0, 5)
if
solve(initial_state):
    print("The puzzle is solvable!")
else:
    print("The puzzle is unsolvable.")
```

Output:



1 2 3

5 6 0

7 8 4

1 2 3

5 0 6

7 8 4

1 2 3

5 8 6

7 0 4

1 2 3

5 8 6

0 7 4

Result: Hence, the simple python program for 8- puzzle is done

Exp: 2 Write the Python Program to solve 8-Queen Problem

Aim:

To write a python program to solve 8-Queen problem.

Algorithm:

Step1: Start with an empty board of size 8x8.

Step2: Place the first queen in the first row and the first column.

Step3: For each subsequent row, check all possible columns to place the queen, and backtrack if a valid position cannot be found.



Step4: A valid position is a column where no other queens are in the same row, column, or diagonal.

Step5: If no solution is found, return failure.

Program:

```
def solve_queens(n):
    board = [-1] * n    result
    = []    def is_valid(row,
    col):        for i in
    range(row):
        if board[i] == col or \
        abs(row - i) == abs(col - board[i]):
            return False
    return True    def
    backtrack(row):
    if row == n:
        result.append(list(board))
    return        for col in range(n):
        if is_valid(row, col):
            board[row] = col
            backtrack(row + 1)
            board[row] = -1
    backtrack(0)    return result
solutions = solve_queens(8)
for solution in solutions:    for
    row in range(8):        line = ""
    for col in range(8):            if
    solution[row] == col:
```



```
        line += "Q "  
else:      line  
+= "- "  
print(line)  
print("\n")
```

Output:

```
[1, 0, 0, 0, 0, 0, 0, 0]  
[0, 0, 0, 0, 1, 0, 0, 0]  
[0, 0, 0, 0, 0, 0, 0, 1]  
[0, 0, 0, 0, 0, 1, 0, 0]  
[0, 0, 1, 0, 0, 0, 0, 0]  
[0, 0, 0, 0, 0, 0, 1, 0]  
[0, 1, 0, 0, 0, 0, 0, 0]  
[0, 0, 0, 1, 0, 0, 0, 0]
```

Result: Hence, the simple python program for 8- Queen problem is done

Exp:3 Write the Python Program for Water Jug Problem

Aim:

To write a python program for water jug problem.

Algorithm:

Step1: Start with two empty jugs of different capacities, and a target quantity of water to measure.

Step2: Fill one jug to its maximum capacity.

Step3: Pour the water from the full jug into the other jug until it is full or the first jug is empty.

Step4: If the second jug is full, pour out the water to empty it.

Step5: Repeat steps 2-4, filling and pouring water between the two jugs, until the desired quantity of water is measured or all possible combinations are tried.



Program:

```
from collections import deque

def water_jug_problem(x, y, z):
    """
    Solves the water jug problem using breadth-first search.
    x: size of jug 1    y: size
    of jug 2    z: target amount
    of water

    Returns the sequence of steps to reach the target amount of water.
    """

    queue = deque([(0, 0, [])]) # Start with both jugs empty
    visited = set()    while queue:
        a, b, steps = queue.popleft()
        if (a, b) in visited:
            continue
        visited.add((a, b))    if
        a == z or b == z:
            return steps

        queue.append((x, b, steps + ['fill jug 1']))
        queue.append((a, y, steps + ['fill jug 2']))
        queue.append((0, b, steps + ['empty jug 1']))
        queue.append((a, 0, steps + ['empty jug 2']))        amount
        = min(a, y - b)

        queue.append((a - amount, b + amount, steps + ['pour jug 1 into jug 2']))
        # Pour jug 2 into jug 1
        amount = min(x - a, b)

        queue.append((a + amount, b - amount, steps + ['pour jug 2 into jug 1']))

    return None
```



```
steps = water_jug_problem(4, 3, 2) if
```

```
steps:
```

```
    print('\n'.join(steps)) else:
```

```
    print('No solution found.')
```

Output:

```
Path of states by jugs followed is :
0 , 0
0 , 3
3 , 0
3 , 3
4 , 2
0 , 2
```

Result: Hence, the simple python program for water jug problem is done

Exp: 4 Write the Python Program for Crypt-Arithmetic Problem.

Aim:

To write a python program for Crypt-Arithmetic problem.

Algorithm:

Step 1: Write down the arithmetic problem in full, including any carry-overs.

Step 2: Replace each letter with a unique digit from 0 to 9.

Step 3: Generate all possible combinations of digits for each letter, subject to any constraints ' from the carry-overs and the rules of arithmetic.

Step 4: For each combination of digits, evaluate the arithmetic problem and check if it satisfies the given constraints.

Step 5: If a solution is found, output the values of the letters and the arithmetic problem, and exit the algorithm.

Step 6: If no solution is found, output a message indicating that no solution exists.

Program:

```
def solve_cryptarithmic(puzzle):
```



"""

Solves the cryptarithmic puzzle.

puzzle: a string containing the puzzle in the form of "WORD + WORD = RESULT"

Returns a dictionary mapping letters to digits, or None if no solution is found.

"""

```
words = puzzle.split()
```

```
letters = set("".join(words))    if
```

```
len(letters) > 10:
```

```
    return None # More than 10 letters, no solution is possible
```

```
for perm in permutations(range(10), len(letters)):
```

```
    mapping = dict(zip(letters, perm))
```

```
    if all(mapping[word[0]] != 0 for word in words): # Check for leading zeros
```

```
    a = sum(mapping[c] * (10 ** (len(word) - i - 1)) for word in words for i, c in
        enumerate(word[::-1]))
```

```
    b = a // 10        a %= 10
```

```
    c = sum(mapping[c] * (10 ** (len(words[2]) - i - 1)) for i, c in enumerate(words[2][::-1]))
```

```
    if a == mapping[words[2][-1]] and b + c == sum(mapping[c] * (10 ** (len(word) - i - 1))
        for word in words[::-1]):        return mapping    return None # Example usage
```

```
puzzle = "SEND + MORE = MONEY"
```

```
mapping = solve_cryptarithmic(puzzle) if
```

```
mapping:
```

```
    print(mapping)
```

```
    print(puzzle.translate(str.maketrans(mapping))) else:
```

```
    print('No solution found.')
```

Output:

After defining the solve_cryptarithmic function, the code provides an example usage using a cryptarithmic puzzle: "SEND + MORE = MONEY." It calls the solve_cryptarithmic function with the puzzle string and stores the result in the mapping variable.



If a solution is found (i.e., mapping is not None), it prints the mapping dictionary and the puzzle string with the letters replaced by their corresponding digits using the translate method. If no solution is found, it prints "No solution found."

Result:

The output has verified.

Exp :5 Write the python program for Missionaries Cannibal problem.

Aim:

To write a python program for Missionaries Cannibal problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.



Program:

```
from typing import List, Tuple from collections import deque

def is_valid_state(state: Tuple[int, int, int, int, int, int]) -> bool:
    """
    Check if a state is valid according to the problem constraints.
    """
    m1, c1, b, m2, c2, _ = state
    if m1 < 0 or c1 < 0 or m2 < 0 or c2 < 0:
        return False
    if (m1 > 0 and m1 < c1) or (m2 > 0 and m2 < c2):
        return False
    return True

def get_successors(state: Tuple[int, int, int, int, int, int]) -> List[Tuple[int, int, int, int, int, int]]:
    """
    Generate all possible valid states that can be reached from a given state.
    """
    m1, c1, b, m2, c2, d = state
    successors = []
    if d == 1:
        # boat is on the left side
        for i in range(3):
            for j in range(3):
                if i+j >= 1 and i+j <= 2:
                    new_state = (m1-i, c1-j, 0, m2+i, c2+j, 1)
                    if is_valid_state(new_state):
                        successors.append(new_state)
    else:
        # boat is on the right side
        for i in range(3):
            for j in range(3):
                if i+j >= 1 and i+j <= 2:
                    new_state = (m1+i, c1+j, 1, m2-i, c2-j, 0)
                    if is_valid_state(new_state):
                        successors.append(new_state)
```



```
range(3):          if i+j >= 1
and i+j <= 2:
    new_state = (m1+i, c1+j, 1, m2-i, c2-j, 0)
if is_valid_state(new_state):
    successors.append(new_state)

return successors

def breadth_first_search() -> List[Tuple[int, int, int, int, int, int]]:
    """
    Find the solution using a breadth-first search algorithm.
    """
    initial_state = (3, 3, 1, 0, 0, 1)
    goal_state = (0, 0, 0, 3, 3, 0)    visited
    = set()
    queue = deque([(initial_state, [])])
    while queue:
        state, path = queue.popleft()    if
        state == goal_state:            return path +
        [state]    visited.add(state)    for
        successor in get_successors(state):
            if successor not in visited:
                queue.append((successor, path+[state]))
    return [] if __name__ == '__main__':
        solution = breadth_first_search()
        print("Solution:")    for state in
        solution:
            print(state)
```



Output:

```
Game Start
Now the task is to move all of them to right side of the river
rules:
1. The boat can carry at most two people
2. If cannibals num greater than missionaries then the cannibals
   would eat the missionaries
3. The boat cannot cross the river by itself with no people on board

M M M C C C |    --- |

Left side -> right side river travel
Enter number of Missionaries travel =>
Invalid input please retry !!
```

Result:

The program was executed and the output was found to be correct.

Exp: 6 Write the python program for Vacuum Cleaner Problem.

Aim:

To write a python program for vacuum cleaner problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or



dirty).

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).

Step 5: Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program:

```
from typing import List, Tuple

def get_successors(state: Tuple[Tuple[int, int], List[List[int]]]) -> List[Tuple[Tuple[int, int],
List[List[int]]]]: """
    Generate all possible valid states that can be reached from a given state.
    """
    position, grid = state    successors = []
    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        x, y = position
        new_x, new_y = x + dx, y + dy        if 0 <= new_x <
len(grid) and 0 <= new_y < len(grid[0]):
            new_grid = [row[:] for row in grid]
            if new_grid[new_x][new_y] == 1:
                new_grid[new_x][new_y] = 0
            successors.append(((new_x, new_y), new_grid))    return successors
def depth_first_search(state: Tuple[Tuple[int, int], List[List[int]]], visited: set) -> bool:
    """
    Find the solution using a depth-first search algorithm.
```



```
""" if all(all(cell == 0 for cell in row) for row in
state[1]):
    return True    visited.add(state)    for
successor in get_successors(state):        if
successor not in visited:            if
depth_first_search(successor, visited):
    return True
return False if __name__ == '__main__':    grid =
[[0, 1, 1, 1],        [0, 0, 1,
0],
        [1, 0, 0, 1],        [1, 1, 1, 0]]
initial_state = ((0, 0), grid)    if
depth_first_search(initial_state, set()):
print("The room can be cleaned.")    else:
    print("The room cannot be cleaned.")
```

Result:

```
All the room are dirty
[[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
Before cleaning the room I detect all of these random dirts
[[1, 0, 1, 1], [0, 0, 0, 1], [1, 0, 0, 0], [0, 0, 0, 0]]
Vaccum in this location now, 0 0
cleaned 0 0
Vaccum in this location now, 0 2
cleaned 0 2
Vaccum in this location now, 0 3
cleaned 0 3
Vaccum in this location now, 1 3
cleaned 1 3
Vaccum in this location now, 2 0
cleaned 2 0
Room is clean now, Thanks for using : A.SAFARJI CLEANER
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
performance= 68.75 %
```

Result:

Hence, the simple python program for Vacuum Cleaner Problem was done



Exp: 7 Write the Python Program to implement BFS

Aim:

To write a python program to implement BFS.

Algorithm:

Step 1: Initialize a queue data structure and a set to keep track of visited nodes.

Step 2: Add the starting node to the queue and to the set of visited nodes.

Step 3: While the queue is not empty, dequeue the next node from the queue.

Step 4: For each neighbor of the dequeued node, if it has not been visited yet, add it to the queue and mark it as visited.

Step 5: Repeat steps 3-4 until the queue is empty.

Program:

```
from collections import deque def
bfs(adj_list, start, end):
    """
    Perform a breadth-first search in the given graph to find the shortest path
    between the start and end nodes.
    """
    queue = deque([(start, [start])])
    visited = set()    while queue:
        node, path = queue.popleft()
        if node == end:    return
        path    if node in visited:
            continue
        visited.add(node)    for
        neighbor in adj_list[node]:
            if neighbor not in visited:
```



```
        queue.append((neighbor, path + [neighbor]))

return None if __name__ == '__main__':

    graph = {0: [1, 2],
              1: [0, 2, 3],
              2: [0, 1, 3],
              3: [1, 2, 4],
              4: [3]}

    start_node = 0
    end_node = 4

    shortest_path = bfs(graph, start_node, end_node)

    if shortest_path is not None:
        print(f"The shortest path between {start_node} and {end_node} is {shortest_path}")
    else:
        print(f"There is no path between {start_node} and {end_node}")
```

Output

```
The Breadth First Search Traversal for The Graph is as Follows:
3 1
```

Result:

Thus, the was executed and found the output was found to be correct.

Exp: 8 Write the Python Program to Implement DFS.

Aim:

To write a python program to implement DFS.



Algorithm:

Step 1: Initialize a set to keep track of visited nodes.

Step 2: Call the DFS-Visit function with the starting node and the set of visited nodes.

Step 3: In the DFS-Visit function, add the current node to the set of visited nodes.

Step 4: For each neighbor of the current node, if it has not been visited yet, recursively call the DFS-Visit function with the neighbor node and the set of visited nodes.

Step 5: Repeat steps 3-4 until all reachable nodes have been visited.

Program:

```
# Define the graph as an adjacency list graph
```

```
= {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['D', 'E'],
```

```
    'C': ['F'],
```

```
    'D': [],
```

```
    'E': ['F'],
```

```
    'F': []
```

```
}
```

```
# Define the DFS function def
```

```
dfs(graph, start, visited=None):
```

```
if visited is None:    visited =
```

```
set()
```

```
    visited.add(start) # mark the node as visited
```

```
for neighbor in graph[start]:    if neighbor
```

```
not in visited:
```

```
    dfs(graph, neighbor, visited) # recursively visit unvisited neighbors
```

```
return visited
```

```
# Call the DFS function with starting node 'A' print(dfs(graph,
```

```
'A'))
```

Output



A D H C F J G K B E I

Result:

Thus, the Program was executed and output found to be correct.

Exp: 9 Write the Python to Implement Travelling Salesman Problem.

Aim:

To write a python program to implement travelling salesman problem.

Algorithm:

Step 1: Initialize the best distance and best route to be infinity and an empty list, respectively.

Step 2: Generate every possible permutation of the cities.

Step 3: For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.

Step 4: Add the distance from the last city to the first city.

Step 5: If the calculated distance is shorter than the current best distance, update the best distance and best route.

Step 6: Return the best distance and best route.

Program:

```
import numpy as np
def tsp(cities):
    # Create a distance matrix
    n = len(cities)
    dist_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i != j:
                dist_matrix[i][j] = np.linalg.norm(cities[i] - cities[j])
```



```
# Initialize variables

tour = [0]

unvisited_cities = set(range(1, n))

total_distance = 0

# Find the nearest unvisited city and add it to the tour
while unvisited_cities:

    current_city = tour[-1]

    nearest_city = min(unvisited_cities, key=lambda city: dist_matrix[current_city][city])

    tour.append(nearest_city)    unvisited_cities.remove(nearest_city)

    total_distance += dist_matrix[current_city][nearest_city]

# Complete the tour by returning to the starting city
tour.append(0)

total_distance += dist_matrix[tour[-2]][0]

return tour, total_distance
```

Output:

```
Path found: ['A', 'F', 'G', 'I', 'J']
```

Result:

Thus the Program was executed and the output was found to be Correct.



Exp 9: Write the python program to implement A* algorithm.

Aim:

To write a python program to implement A* algorithm.

Algorithm:

Step1 :Initialize the open list with the starting node and the closed list as an empty set. Step

2: Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

Step 3: While the open list is not empty, select the node with the lowest f score and set it as the current node.

Step 4: If the current node is the goal node, return the reconstructed path.

Remove the current node from the open list and add it to the closed list.

Step 5: For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

Step 6: If the neighbor is already in the closed list, skip it.

Step 7: If the neighbor is not in the open list, add it and set its g score and f score.

Step 8 :If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

Step 9 : Otherwise, update the came from dictionary, g score, and f score of the neighbor.

Step 10 : Repeat steps 3-10 until the open list is empty.

If the goal node was never reached, return failure.

Program:

```
import heapq
```

```
def astar(start, goal, neighbors_fn, heuristic_fn):
```

```
    """
```

```
    Find the shortest path from start to goal using A* algorithm.
```



Arguments:

start: the starting node goal: the goal node neighbors_fn: a
function that takes a node and returns its neighbors heuristic_fn: a
function that takes a node and returns the estimated
distance to the goal node

Returns:

A tuple containing the path from start to goal and its cost

"""

```
frontier = []
```

```
heapq.heappush(frontier, (0, start))
```

```
came_from = {}    cost_so_far = {}
```

```
came_from[start] = None
```

```
cost_so_far[start] = 0
```

```
while frontier:
```

```
    _, current = heapq.heappop(frontier)
```

```
    if current == goal:
```

```
        break
```

```
    for neighbor in neighbors_fn(current):      new_cost =
```

```
cost_so_far[current] + 1      if neighbor not in cost_so_far or new_cost
```

```
< cost_so_far[neighbor]:
```

```
        cost_so_far[neighbor] = new_cost
```

```
priority = new_cost + heuristic_fn(neighbor, goal)
```

```
heapq.heappush(frontier, (priority, neighbor))
```

```
came_from[neighbor] = current
```

```
path = [goal]    current = goal
```

```
while current != start:      current
```



```
= came_from[current]  
path.append(current)  
path.reverse()  
  
return path, cost_so_far[goal]
```

Output:

```
# Print the result print("Shortest  
path:", path) print("Cost:", cost)
```

In this example, we have a 5x5 grid where 0 represents passable cells and 1 represents obstacles. The `grid_neighbors` function defines the neighbors of a node on the grid, considering only right, left, up, and down movements. The `grid_heuristic` function calculates the Manhattan distance between two nodes on the grid.

By calling the `astar` function with the start and goal nodes, along with the `grid_neighbors` and `grid_heuristic` functions, we can find the shortest path from the start (0, 0) to the goal (4, 4) on the grid. The resulting path and cost are then printed.

Result:

Thus the Program was executed and the output was found to be Correct.

Exp 10 : Write the python program for Map Coloring to implement CSP.

Aim:

To write a python program for map coloring to implement CSP.

Algorithm:

Initialize an empty assignment for the map coloring problem.



For each country in the map, add a variable X_i to the assignment.

Define the constraints as pairs of adjacent countries, and add a constraint (X_i, X_j) to the CSP such that $X_i \neq X_j$.

Define the domain of each variable to be the set of available colors.

Call the Backtrack function to find a consistent assignment for the variables.

In the Backtrack function, check if the assignment is complete. If it is, return the assignment.

Select an unassigned variable X_i from the assignment.

For each value v in the domain of X_i , check if it is consistent with the constraints and the assignment so far.

If it is, assign $X_i = v$ to the assignment and recursively call Backtrack with the new assignment.

If the result of Backtrack is not a failure, return the result.

If the result is a failure, remove the assignment $X_i = v$ and try the next value in the domain.

If all values in the domain have been tried and failed, return failure.

Program:

```
from typing import Dict, List from
itertools import product from
copy import deepcopy class CSP:
    def __init__(self, variables:
        List[str], domains: Dict[str,
        List[str]], constraints):
        self.variables = variables
        self.domains = domains
        self.constraints = constraints

    def solve(self):    assignments = {}
    return self.backtrack(assignments)
```



```
def backtrack(self, assignments):    if
len(assignments) == len(self.variables):
    return assignments

    var = self.select_unassigned_variable(assignments)
for value in self.order_domain_values(var, assignments):
    new_assignments = deepcopy(assignments)
    new_assignments[var] = value    if
self.is_consistent(new_assignments):
    result = self.backtrack(new_assignments)
    if result is not None:
        return result

return None

def select_unassigned_variable(self, assignments):
for var in self.variables:

    if var not in assignments:
        return var

def order_domain_values(self, var, assignments):
    values = self.domains[var]
    return sorted(values, key=lambda value: self.get_num_conflicts(var, value, assignments))

def get_num_conflicts(self, var, value, assignments):
    conflicts = 0    for constraint in
self.constraints:    if var in constraint and
len(constraint) == 2:
```



```
        other_var = constraint[0] if constraint[1] == var else constraint[1]
        if other_var in assignments and assignments[other_var] == value:
            conflicts += 1
    return conflicts
```

```
def is_consistent(self, assignments):
    for constraint in self.constraints:
        if all(var in assignments for var in constraint):
            if not self.satisfies_constraint(assignments, constraint):
                return False
    return True
```

```
def satisfies_constraint(self, assignments, constraint):
    if len(constraint) == 2:
        val1, val2 = assignments[constraint[0]], assignments[constraint[1]]
        return val1 != val2
    else:
        return True
```

```
def map_coloring():
    variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
    domains = {
        'WA': ['r', 'g', 'b'],
        'NT': ['r', 'g', 'b'],
        'SA': ['r', 'g', 'b'],
        'Q': ['r', 'g', 'b'],
        'NSW': ['r', 'g', 'b'],
        'V': ['r', 'g', 'b'],
    }
```



```
'T': ['r', 'g', 'b']
}
constraints = [
('WA', 'NT'),
('WA', 'SA'),
('NT', 'SA'),
('NT', 'Q'),
('SA', 'Q'),
('SA', 'NSW'),
('SA', 'V'),
('Q', 'NSW'),
('NSW', 'V')
]
csp = CSP(variables, domains, constraints)
result = csp.solve()  if result is not None:

    print(result)
else:
    print("No solution found")
```

map_coloring() **output:**

```
Solution Exists: Following are the assigned colors
1  2  3  2
```

Result:

Hence, the simple python program for Map Coloring to implement CSP is done



Exp 11: Write the python program for Tic Tac Toe game

Program:

```
# Tic Tac Toe game
```

```
board = [' ' for _ in range(9)]
```

```
def print_board():
```

```
    row1 = '|'.join(board[0:3])
```

```
    row2 = '|'.join(board[3:6])    row3
```

```
    = '|'.join(board[6:9])
```

```
    print(row1)
```

```
        print('-' * 5)
```

```
        print(row2)
```

```
        print('-' * 5)
```

```
        print(row3)
```

```
def player_move(icon):
```

```
    if icon == 'X':
```

```
        player = 1    else:
```

```
            player = 2
```

```
    print("Player {}'s turn".format(player))
```

```
    choice = int(input("Enter your move (1-9): ").strip())
```

```
    if board[choice-1] == ' ':    board[choice-1] = icon
```



else:

print("That space is already taken!")

def is_victory(icon):

```
    if (board[0] == icon and board[1] == icon and board[2] == icon) or \
        (board[3] == icon and board[4] == icon and board[5] == icon) or \
        (board[6] == icon and board[7] == icon and board[8] == icon) or \
        (board[0] == icon and board[3] == icon and board[6] == icon) or \
        (board[1] == icon and board[4] == icon and board[7] == icon) or \
        (board[2] == icon and board[5] == icon and board[8] == icon) or \
        (board[0] == icon and board[4] == icon and board[8] == icon) or \
        (board[2] == icon and board[4] == icon and board[6] == icon):
        return True
```

else:

return False

def is_draw():

```
    if ' ' not in board:
        return True
    else:
        return False
```

while True:

print_board()

player_move('X')

if is_victory('X'):

print("X Wins! Congratulations!")

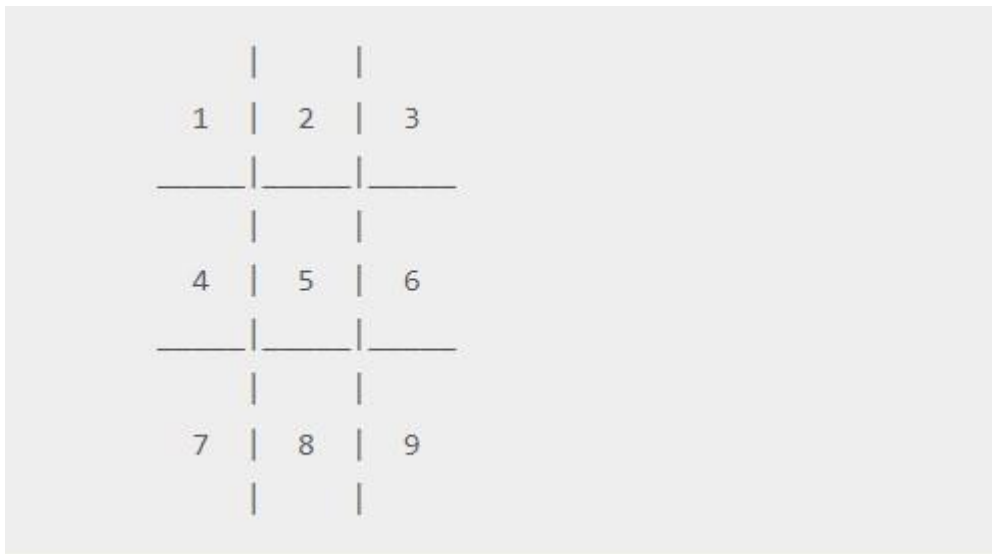
break elif

is_draw():



```
print("It's a Draw!")  
  
break  
  
print_board()  
  
player_move('O')  
  
if is_victory('O'):  
    print("O Wins! Congratulations!")  
    break elif  
  
is_draw():  
    print("It's a Draw!")  
  
break
```

output:



Result:

Hence, the simple the python program for Tic Tac Toe game is done.



Exp 12 : Write the python program to implement Minimax algorithm for gaming.

Program:

```
# Define a function to evaluate the score of a game state def
evaluate(state):

    # This function should return a score that represents how good the state is for the current
    player

    # A positive score means the current player is winning, and a negative score means the
    current player is losing

# Define the Minimax algorithm function def
minimax(state, depth, player):

    # If the game is over or the maximum depth has been reached, return the score of the
    current state    if depth == 0 or game_over(state):
        return evaluate(state)

    # If it's the current player's turn, maximize their score
    if player == "X":
        best_score = float('-inf')
        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = minimax(new_state, depth - 1, "O")
            best_score = max(best_score, score)    return
        best_score

    # If it's the other player's turn, minimize their score
    else:
        best_score = float('inf')
```



```
for move in get_possible_moves(state):  
    new_state = make_move(state, move, player)  
    score = minimax(new_state, depth - 1, "X")  
    best_score = min(best_score, score)    return  
best_score
```

output:

```
root = TreeNode(0)  
root.left = TreeNode(3)  
root.right = TreeNode(0)  
root.right.left = TreeNode(0)  
root.right.right = TreeNode(0)  
root.right.left.left = TreeNode(-3)  
root.right.right.right = TreeNode(4)
```

Result:

Hence, the simple the python program to implement Minimax algorithm for gaming is done